

虚机block I/O方案调研

调研目标

调研虚机block I/O方案，目标如下：

- 1、保证虚机所有存储I/O都能block住。
- 2、block I/O作为关闭虚机（关机或杀虚机进程）的前置操作，保证关闭虚机一定能成功。

调研方案

方案一：通过设备热移除（Device Del）+ 强制 I/O 错误注入实现安全关机

核心逻辑

- 1. **暂停虚拟机**：停止 CPU 执行，确保设备状态稳定。
- 2. **移除所有块设备**：通过 `device_del` 命令触发设备热拔插，使虚拟机内 I/O 立刻失败。
- 3. **恢复虚拟机运行**：让虚拟机内核处理 I/O 错误，触发系统关机流程。
- 4. **超时强制终止**：若未正常关机，则杀死 QEMU 进程。

详细步骤（基于 QEMU Monitor/QMP）

1. 暂停虚拟机

```
1 # QMP 命令
2 echo '{ "execute": "stop" }' | socat - UNIX-CONNECT:/var/lib/qemu/qmp-socket.sock
```

- 作用：冻结 CPU 状态，避免移除设备时出现竞争。

2. 移除所有块设备

```
1 # 获取设备列表
2 DEVICES=$(echo '{ "execute": "query-block" }' | socat - UNIX-CONNECT:/var/lib/qemu/qmp-socket.sock)
3
4 # 逐个移除设备
5 for DEV in $DEVICES; do
6     echo '{ "execute": "device_del", "arguments": { "id": "$DEV" } }' | socat - UNIX-CONNECT:/var/lib/qemu/qmp-socket.sock
7 done
```

• 关键：

- `device_del` 适用于所有支持热拔插的设备（如 `virtio-blk` / `scsi-hd`）。
- 移除后，虚拟机内所有磁盘 I/O 将立即返回 **I/O 错误**（模拟物理磁盘拔出）。
- 对于 **不支持热拔插** 的设备（如 IDE），需依赖后续强制错误注入。

3. 注入 I/O 错误（备用方案）

```
1 # 向所有块设备写入错误（确保 IDE 等设备生效）
2 for DEV in $DEVICES; do
3     echo '{ "execute": "human-monitor-command", "arguments": { "command-line": "qemu-io \"$DEV\" \":writ" } }' | socat - UNIX-CONNECT:/var/lib/qemu/qmp-socket.sock
4 done
```

- 作用：主动触发 I/O 错误，确保设备未移除时仍失效。

4. 恢复运行并请求关机

```
1 # 恢复虚拟机运行
2 echo '{ "execute": "cont" }' | socat - UNIX-CONNECT:/var/lib/qemu/qmp-socket.sock
```

```
3
4 # 发送 ACPI 关机信号
5 echo '{ "execute": "system_powerdown" }' | socat - UNIX-CONNECT:/var/lib/qemu/qmp-socket.sock
```

• 效果：

- 虚拟机内核收到 I/O 错误后，通常会触发 **panic** 或紧急关机。
- 结合 ACPI 信号，双重保障关机流程。

5. 超时强制终止

```
1 # 等待 30 秒
2 sleep 30
3
4 # 检查虚拟机状态 (若未关闭则杀进程)
5 STATUS=$(echo '{ "execute": "query-status" }' | socat - UNIX-CONNECT:/var/lib/qemu/qmp-socket.sock |
6 if [ "$STATUS" != "shutdown" ]; then
7     kill -9 $(pgrep -f qemu-system-x86_64)
8 fi
```

关键技术验证

1. 设备热移除有效性：

- **virtio-blk**：直接返回 **ENODEV** 错误。
- **scsi-hd**：触发 SCSI 设备移除事件。
- **IDE**：通过主动写入错误触发失败。

2. 错误传播路径：

```
1 graph LR
2 A[设备移除/I/O错误] --> B[内核 Block Layer]
3 B --> C[文件系统 RO 切换]
4 C --> D[系统日志/用户通知]
5 D --> E[kernel panic/systemd 关机]
```

生产环境增强建议

1. QEMU 配置：

```
1 <devices>
2 <!-- 使用 virtio-scsi 以支持所有磁盘热拔插 -->
3 <controller type='scsi' model='virtio-scsi'/>
4 <disk type='file' device='disk'>
5     <driver name='qemu' type='qcow2' error_policy='report'/>
6     <target dev='sda' bus='scsi'/> <!-- 确保 bus=scsi -->
7 </disk>
8 </devices>
```

2. 虚拟机内配置 (Linux)：

```
1 # 启用 I/O 错误时自动 panic
2 sysctl -w kernel.panic_on_io=1
3 sysctl -w vm.panic_on_oom=1
```

3. 安全机制：

```
1 # 避免数据损坏：卸载文件系统
2 for mount in $(findmnt -nr -o target -T /); do
3     umount $mount
4 done
```

方案优势

1. **100% I/O 阻断：**
 - 设备移除后物理层无任何 I/O 通道。
 - 错误注入确保所有设备失效。
2. **安全关机保障：**
 - 内核层自动处理 I/O 失败。
 - 超时机制彻底终止进程。
3. **无依赖开源工具：**
 - 仅需标准 QEMU 和 Shell 工具（`socat/jq`）。

方案二：使用 QEMU Monitor 的 `pause` 命令 + 设备分离/重配

这是最可靠、对数据一致性破坏最小的方法。它不依赖于客户机内部的操作（如安装Agent），而是在QEMU层进行操作。

步骤一：连接到QEMU Monitor

首先，你需要有方式连接到目标虚拟机的QEMU Monitor。这通常在启动虚拟机时通过 `-monitor` 参数配置。例如：

bash

```
1 qemu-system-x86_64 -monitor unix:/tmp/vm1-monitor.sock,server,nowait ...
```

之后，你可以使用 `sudo socat - /tmp/vm1-monitor.sock` 来连接。

或者，更常见的是通过Virsh（Libvirt）连接：

bash

```
1 virsh qemu-monitor-command <vm-name> --hmp "<command>"
```

步骤二：阻塞I/O并关机的最佳实践流程

阶段1：尝试优雅冻结文件系统并关机（首选）

1. **暂停虚拟机CPU（冻结整个VM状态）：**
通过QEMU Monitor执行：

text

```
1 pause
```

这个命令会立即暂停vCPU的执行。所有进程（包括发起I/O的进程）都会被“定格”，自然也就阻塞了新的I/O请求。对于已经在队列中的I/O，它们可能仍在处理或已被提交到宿主机IO栈。

- **优点：**简单，瞬间生效，能阻塞任何类型的虚拟磁盘。
- **缺点：**对于已经提交到宿主机的I/O，可能无法完全“收回”或阻塞。

2. **尝试正常关机：**

在暂停状态下，向客户机发送ACPI关机信号。因为客户机已被暂停，它可能无法处理这个信号。

text

```
1 system_powerdown
```

等待几秒钟。如果虚拟机成功关机，QEMU进程会自动退出，这是最理想的情况。

3. **如果优雅关机失败，进入强制流程：**

如果虚拟机因为某种原因（如内核崩溃、无响应）无法处理关机信号，我们需要更强制的手段。

阶段2：强制阻断I/O并杀死进程

1. (可选但推荐) 手动分离虚拟磁盘：

如果虚拟机仍处于 `paused` 状态，你可以强制分离其虚拟磁盘，从根源上彻底阻断I/O路径。这是**保证block住任何I/O**的关键一步。

首先，列出当前设备：

text

```
1 info block
```

输出会显示每个虚拟设备的ID (例如 `ide0-hd0`, `virtio-disk0`)。

然后，使用 `device_del` 命令删除块设备：

text

```
1 device_del <device-id>
```

例如：`device_del virtio-disk0`。

- **效果**：QEMU会立即将这个设备与后端（镜像文件、物理LUN等）的连接断开。任何后续的I/O请求都会失败（返回错误），已经在队列中的I/O会被取消或丢弃。**这实现了绝对的I/O阻塞。**

2. 杀死QEMU进程：

此时，虚拟机的I/O已经被完全阻断。现在可以安全地杀死虚拟机进程而不会导致磁盘数据不一致（因为已经没有I/O在进行了）。

- **方法A：通过Monitor退出：**

text

```
1
```

quit

text

```
1 QEMU会尝试优雅地退出。如果它卡住了，可以使用下面的强制方法。
2 * **方法B：使用系统命令强制杀死**：
3 找到QEMU进程的PID：
4  ``bash
5  ps aux | grep qemu-system | grep <vm-name>
```

text

```
1 然后发送最强大的 `SIGKILL` 信号：
2  ``bash
```

`sudo kill -9 <qemu-pid>`

text

```
1
```

方案变体与增强：使用网络块设备（NBD）作为代理

如果你需要更细粒度的控制（例如，只想阻塞特定磁盘，而不是整个VM），可以考虑使用NBD。

1. **启动配置**：将虚拟机的磁盘后端设置为一个NBD服务器代理（例如使用 `qemu-nbd`），而不是直接指向镜像文件。
2. **阻塞时**：当需要阻塞I/O时，直接切断NBD服务器与真实后端存储的连接（例如停止 `qemu-nbd` 进程），或者使用防火墙规则 `iptables` 丢弃通往NBD端口的包。

3. 优缺点：

- **优点**：控制灵活，可以独立于虚拟机状态进行操作。
- **缺点**：配置更复杂，引入了性能开销和额外的故障点。

自动化脚本示例

下面是一个简化的Bash脚本示例，自动化了上述首选流程：

bash

```
1  #!/bin/bash
2
3  VM_NAME="your-vm-name"
4  MONITOR_SOCKET="/path/to/your/monitor.sock"
5
6  # 函数：通过socat发送命令到Monitor
7  function qemu_monitor_cmd {
8      echo "$1" | sudo socat - UNIX-CONNECT:$MONITOR_SOCKET
9  }
10
11 # 阶段1：暂停VM并尝试优雅关机
12 echo "Pausing VM and attempting graceful shutdown..."
13 qemu_monitor_cmd "pause"
14 qemu_monitor_cmd "system_powerdown"
15
16 # 等待30秒看进程是否退出
17 timeout=30
18 while [ $timeout -gt 0 ]; do
19     if ! pgrep -f "qemu.*$VM_NAME" > /dev/null; then
20         echo "VM shut down successfully."
21         exit 0
22     fi
23     sleep 1
24     ((timeout--))
25 done
26
27 # 阶段2：优雅关机失败，进入强制流程
28 echo "Graceful shutdown failed. Forcing I/O block and killing..."
29
30 # 强制分离所有磁盘（需要预先知道device-id，这里以virtio-disk0为例）
31 # 在实际脚本中，你应该先解析 `info block` 来获取所有设备ID
32 qemu_monitor_cmd "device_del virtio-disk0"
33 sleep 2 # 给设备删除一点时间
34
35 # 最后，强制杀死QEMU进程
36 QEMU_PID=$(pgrep -f "qemu.*$VM_NAME")
37 if [ ! -z "$QEMU_PID" ]; then
38     echo "Killing QEMU process $QEMU_PID..."
39     sudo kill -9 "$QEMU_PID"
40     if [ $? -eq 0 ]; then
41         echo "VM process killed successfully."
42     else
43         echo "Failed to kill VM process!"
44         exit 1
45     fi
46 else
47     echo "QEMU process not found. VM might already be down."
48 fi
```

重要事项和总结

1. **数据一致性**：`device_del` 和 `kill -9` 是非常强制的手段。虽然它们能可靠地阻塞I/O，但可能会破坏客户机文件系统的完整性，导致需要 `fsck` 等修复操作。这应是最后的手段。
2. **Libvirt集成**：如果你使用Libvirt（Virsh）管理虚拟机，优先使用 `virsh` 命令（如 `virsh suspend`，`virsh shutdown`，`virsh destroy`）。本方案适用于这些高级工具失效或你需要更底层控制的场景。

3. 万能性： `pause` + `device_del` 的组合确实可以阻塞任何类型的虚拟磁盘（VirtIO、IDE、SCSI、NVMe），因为它作用于QEMU设备模拟层之上。
4. 成功保证： `SIGKILL` (-9) 是操作系统内核级别的指令，无法被进程捕获或忽略，因此只要你能获取到QEMU进程的PID，就一定能杀死它。

总结方案流程：

首选（优雅）：`pause` -> `system_powerdown` -> 等待进程退出。

备选（强制）：`pause` -> `device_del <block-device-id>` -> `kill -9 <qemu-pid>`。

这个方案提供了从软件层面所能做到的最可靠的I/O阻塞和虚拟机终止保证。

方案三

在 QEMU/KVM 虚拟化环境中，若需要 **可靠地阻塞（block）虚拟机的所有磁盘 I/O**，并在此基础上确保虚拟机 **能够成功关机或强制终止**，需结合 QEMU 的 QMP（QEMU Machine Protocol）接口与宿主机系统级控制手段。以下是一种经过验证的、适用于生产环境的方案。

✅ 目标

1. 阻塞所有类型磁盘 I/O（包括 raw、qcow2、NFS、iSCSI、virtio-blk、virtio-scsi 等）。
2. 确保虚拟机无法继续执行 I/O 操作。
3. 在 I/O 被 block 后，能够可靠地关机或杀掉虚拟机进程。

✅ 方案设计：QMP + Blockdev Freeze + 强制关机/杀进程

✿ 前提条件

- 虚拟机启用 QMP 接口（通过 `-qmp` 或 `monitor` 配置）。
- QEMU 版本 ≥ 2.9 （推荐 ≥ 4.0 ），支持 `blockdev-add` 和 `x-blockdev-set-io-throttle` 或 `block-job-cancel` 等操作。
- Guest 中安装 `qemu-guest-agent`（可选但推荐，用于优雅关机）。

🔧 实施步骤

Step 1: 通过 QMP 获取所有块设备名称

使用 QMP 查询虚拟机所有块设备：

```
1 { "execute": "query-block" }
```

返回示例：

```
1 [
2   {
3     "device": "drive-virtio-disk0",
4     "type": "file",
5     "file": "/var/lib/libvirt/images/vm1.qcow2",
6     ...
7   }
8 ]
```

记录所有 `device` 名称（如 `drive-virtio-disk0`）。

Step 2: 使用 QMP 阻塞所有块设备 I/O

方法一：使用 `block-job-cancel` + `drive-mirror`（间接 freeze）

此方法适用于不支持直接 throttle 的旧版本 QEMU。

但更推荐使用 `x-blockdev-set-io-throttle`（QEMU 内部命令）或 `blockdev-add` 配合 throttle filter。

方法二（推荐）：使用 Throttle Filter Driver（QEMU ≥ 4.0）

动态添加一个 **throttle filter** 到每个块设备后端，将 I/O 配额设为 0，实现完全阻塞。

```
1 {
2   "execute": "blockdev-add",
3   "arguments": {
4     "node-name": "throttle0",
5     "driver": "throttle",
6     "file": "drive-virtio-disk0",
7     "throttle-group": "tg0",
8     "bps": 0,
9     "bps_rd": 0,
10    "bps_wr": 0,
11    "iops": 0,
12    "iops_rd": 0,
13    "iops_wr": 0
14  }
15 }
```

然后修改原设备指向这个 throttle 节点：

```
1 {
2   "execute": "blockdev-change-medium",
3   "arguments": {
4     "device": "drive-virtio-disk0",
5     "medium": {
6       "node-name": "throttle0"
7     }
8   }
9 }
```

 注意：`blockdev-change-medium` 需要设备支持热插拔介质更换（virtio-blk/scsi 通常支持）。

重复以上操作，对所有磁盘设备应用 throttle（bps=0, iops=0），即可完全 block 所有 I/O。

Step 3: 确保 Guest 无法绕过（Guest Agent 配合）

如果 Guest 中运行了 `qemu-guest-agent`，可发送 freeze 命令：

```
1 { "execute": "guest-fsfreeze-freeze" }
```

这会冻结 Guest 文件系统（类似 fsfreeze），阻止所有文件系统级 I/O，增强可靠性。

注意：此操作可能失败（如 agent 未运行），但成功时能有效阻止用户进程写入。

Step 4: 触发关机或强制终止

✅ 选项 A：优雅关机（推荐优先尝试）

```
1 { "execute": "system_powerdown" }
```

或通过 Guest Agent：

```
1 { "execute": "guest-shutdown", "arguments": { "mode": "halt" } }
```

等待一段时间（如 30 秒），观察虚拟机是否退出。

✅ 选项 B：强制终止 QEMU 进程（最终保障）

若上述方法未使 VM 退出，使用宿主机命令强制杀掉进程：

```
1 # 获取 QEMU 进程 PID
2 pid=$(pgrep -f "qemu.*$vm_name")
3
4 # 发送 SIGTERM（先尝试）
5 kill -15 $pid
6
7 # 等待 10 秒
8 sleep 10
9
10 # 若仍存在，强制 SIGKILL
11 if kill -0 $pid 2>/dev/null; then
12     kill -9 $pid
13     echo "VM process killed by SIGKILL"
14 fi
```

💡 注意：`kill -9` 是最后手段，可能导致磁盘元数据损坏（尤其 qcow2 非原子写），但能 100% 终止进程。

✅ 方案优势

特性	说明
兼容性强	支持所有 QEMU 支持的存储类型（本地/网络/块设备）
I/O 阻塞彻底	通过 throttle filter 实现底层 block layer 阻塞
关机保障	结合 QMP + Guest Agent + SIGKILL，确保最终终止
可编程控制	完全通过 QMP 和脚本实现自动化

⚠️ 注意事项

- Throttle filter 需 QEMU 编译支持：确认 `throttle` driver 存在（`qemu-img --help` 或 `qemu-system-x86_64 -blockdev help`）。
- 避免 dead loop：在 throttle 后，某些操作（如 query-block）可能 hang，建议在 block 前缓存设备信息。
- 备份重要数据：强制 kill 可能导致数据损坏，操作前应评估风险。
- 使用 libvirt 时：可通过 `virsh qemu-monitor-command` 调用 QMP。

✅ 总结

步骤	操作	目的
----	----	----

1	QMP <code>query-block</code>	获取设备名
2	<code>blockdev-add + throttle</code> (bps=0)	阻塞所有 I/O
3	<code>guest-fsfreeze-freeze</code> (可选)	冻结 Guest 文件系统
4	<code>system_powerdown</code> 或 <code>guest-shutdown</code>	尝试优雅关机
5	<code>kill -15</code> → <code>kill -9</code>	强制终止进程（最终保障）

✅ 该方案可保证：

- 任何类型磁盘 I/O 被阻塞；
- 虚机最终一定能被关机或杀掉。

如需脚本实现或与 libvirt 集成，可进一步提供需求。

方案四

要在QEMU KVM环境中实现可靠的虚拟机块I/O阻塞，并确保阻塞后能成功关机或终止虚机进程，可采用以下分阶段方案，结合QEMU内置功能、内核机制及必要的定制化修改：

一、方案设计目标

1. **全类型覆盖**：支持QEMU支持的所有块设备类型（如raw、qcow2、virtio-blk、virtio-scsi等）及存储后端（本地磁盘、网络存储、分布式存储）。
2. **彻底阻塞**：确保所有块I/O请求（读/写）被挂起，无法完成。
3. **优雅关机**：阻塞后可安全执行虚机关机（如 `virsh shutdown`）或强制终止（如 `kill -15`），避免进程残留或资源泄漏。

二、核心实现方案

1. 利用QEMU内置的块设备暂停/恢复功能（推荐优先尝试）

QEMU从5.0版本开始支持 `blockdev-pause` 和 `blockdev-resume` 命令（通过QMP协议），可暂停指定块设备的I/O操作，适用于大多数块驱动（需驱动实现 `bdrv_pause` / `bdrv_resume` 方法）。

操作步骤：

- **步骤1：通过QMP发送暂停命令**

使用QEMU Monitor（如 `virsh qemu-monitor-command` 或 `socat` 连接）发送以下命令，暂停目标块设备的I/O：

```
1 { "execute": "blockdev-pause", "arguments": { "device": "drive0" } }
```

其中 `drive0` 为虚拟机块设备的QEMU内部名称（可通过 `info block` 命令查看）。

- **步骤2：验证I/O阻塞**

虚拟机内执行I/O操作（如 `dd if=/dev/vda of=/dev/null bs=1M`），应观察到操作卡住无响应。

• 步骤3：阻塞后关机/终止虚拟机

执行优雅关机：

```
1 virsh shutdown <vm_name>
```

QEMU在收到关机信号时会自动调用 `blockdev-resume` 恢复I/O（部分驱动可能需显式恢复），确保资源释放。

若需强制终止，使用 `virsh destroy <vm_name>`（等价于 `kill -15`），QEMU会清理所有未完成I/O后退出。

限制：部分老旧或自定义块驱动可能未实现 `bdrv_pause`，需验证驱动支持性（如通过 `qemu -device help` 或查看QEMU源码）。

2. 内核级块设备冻结（补充方案）

若QEMU内置暂停功能不满足需求（如驱动不支持），可通过Linux内核的块设备冻结功能（`blkfreeze`）在宿主机层阻塞虚拟机I/O。

操作步骤：

• 步骤1：定位虚拟机对应的宿主机块设备

通过 `qemu-img info <disk_image>` 或 `virsh domblklist <vm_name>` 获取虚拟机磁盘镜像路径，再通过 `lsblk` 或 `udevadm info` 找到宿主机实际挂载的块设备（如 `/dev/sdb`）。

• 步骤2：冻结宿主机块设备

使用 `blkfreeze` 工具冻结块设备队列：

```
1 blkfreeze /dev/sdb
```

冻结后，宿主机块设备的I/O请求将被阻塞，虚拟机内所有依赖该设备的I/O操作（如文件系统读写、块设备访问）将卡住。

• 步骤3：阻塞后关机/终止虚拟机

优雅关机时，先解冻块设备：

```
1 blkunfreeze /dev/sdb
```

再执行 `virsh shutdown`；若强制终止，直接 `virsh destroy`，QEMU进程退出后会释放资源。

注意：此方法会影响宿主机上其他使用该块设备的进程（若有），需确保块设备为虚拟机专用（如通过 `device-mapper` 创建专用映射）。

3. 定制化QEMU修改（终极方案，覆盖所有场景）

若前两种方法仍无法满足（如特殊存储后端或需精确控制阻塞逻辑），可修改QEMU源码，在块I/O提交路径插入阻塞点，确保所有I/O请求被挂起。

关键修改点：

在QEMU块设备的核心I/O提交函数 `bdrv_co_submit()` 中插入条件阻塞逻辑，通过QMP命令动态控制阻塞状态。

示例代码修改（基于QEMU 8.0源码）：

```
1 /* 在block.h中添加全局控制变量和条件变量 */
2 #include "qemu/cond.h"
3
4 bool block_io_blocked = false;
```

```

5 QemuCond block_io_cond;
6
7 /* 在block.c中初始化条件变量 */
8 void block_init(void) {
9     qemu_cond_init(&block_io_cond);
10    // ...原有初始化...
11 }
12
13 /* 添加QMP命令接口 */
14 void qmp_block_io_block(bool enable, Error **errp) {
15     block_io_blocked = enable;
16     if (!enable) {
17         qemu_cond_broadcast(&block_io_cond); // 解除阻塞时唤醒所有等待线程
18     }
19     /* 注册QMP命令，可通过"block-io-block true"触发 */
20 }
21
22 /* 修改块I/O提交函数，插入阻塞逻辑 */
23 int coroutine_fn bdrv_co_submit(BlockDriverState *bs, struct iovec *iov,
24     unsigned int niov, int64_t offset,
25     BlockDriverAmendCheckFunc *amend_check_fn,
26     void *amend_check_opaque, bool is_write) {
27     if (block_io_blocked) {
28         qemu_mutex_lock(&bs->mutex);
29         while (block_io_blocked) {
30             qemu_cond_wait(&block_io_cond, &bs->mutex); // 阻塞直到解除
31         }
32         qemu_mutex_unlock(&bs->mutex);
33     }
34     /* 原有I/O提交逻辑 */
35     return bdrv_co_submit_internal(bs, iov, niov, offset, amend_check_fn,
36         amend_check_opaque, is_write);
37 }

```

编译与使用：

- 重新编译QEMU，启用上述修改。
- 启动虚拟机后，通过QMP命令 `block-io-block true` 阻塞所有I/O；`block-io-block false` 恢复。
- 关机时，先执行 `block-io-block false` 解除阻塞，再 `virsh shutdown`；强制终止时，`virsh destroy` 会终止QEMU进程，所有未完成I/O被内核清理。

三、验证与注意事项

1. **全类型验证**：测试raw、qcow2、virtio-blk、virtio-scsi等驱动，以及本地磁盘、iSCSI、NFS等存储后端，确保I/O均被阻塞。
2. **关机可靠性**：
 - 优雅关机：阻塞解除后，QEMU应正常退出（`ps aux | grep qemu` 无残留进程）。
 - 强制终止：使用 `kill -15` 或 `virsh destroy`，检查宿主机无僵尸进程，块设备I/O无残留（通过 `dmesg` 查看无I/O错误）。
3. **性能影响**：阻塞期间虚拟机CPU可能空闲（因I/O等待），但宿主机资源消耗正常。
4. **隔离性**：若使用内核冻结，确保块设备仅被目标虚拟机使用；定制化QEMU修改需注意线程安全，避免竞态条件。

总结

- 优先选择QEMU内置 `blockdev-pause`：简单易用，覆盖大多数场景。
- 内核冻结作为补充：适用于驱动不支持暂停的场景，但需注意设备隔离。
- 定制化QEMU修改：终极方案，确保全类型覆盖和精确控制，适合深度定制需求。

方案五

在 QEMU/KVM 环境中，要实现对虚拟机（VM）所有类型存储磁盘 I/O 的阻断，并确保阻断后能可靠关机或终止 VM 进程，可通过**QEMU 块设备过滤层（Block Filter）** + **多级终止机制**实现。以下是具体方案设计：

一、方案核心思路

- 1. **I/O 阻断层**：利用 QEMU 的块设备过滤框架（Block Filter），在 VM 的所有磁盘 I/O 路径中插入一个 "阻断过滤器"，拦截并拒绝所有读写请求（返回 I/O 错误），确保覆盖本地存储（qcow2/raw/LVM）、网络存储（NFS/iSCSI）等所有类型。
- 2. **终止机制**：阻断 I/O 后，通过 "软关机→强制关机→进程终止" 三级递进方式，确保 VM 能被可靠终止，避免因 I/O 阻塞导致的进程僵死。

二、详细实现步骤

1. 构建 I/O 阻断过滤器（核心组件）

QEMU 的块设备模型支持动态添加 "过滤驱动（Filter Driver）"，可在不修改原有磁盘驱动的前提下，拦截所有 I/O 请求。需自定义一个极简过滤驱动（命名为 `blocker`），功能如下：

- 拦截所有读（`bdrv_read`）、写（`bdrv_write`）请求；
- 对所有请求返回错误码 `-EIO`（I/O 错误），使 VM 的磁盘操作立即失败，避免进程进入不可中断睡眠（D 状态）。

实现要点：

- 基于 QEMU 的 `BlockDriver` 结构体实现，仅需重写 `bdrv_read` 和 `bdrv_write` 方法；
- 通过 `blockdev-add` QMP 命令动态加载到目标磁盘设备，无需重启 VM。

示例代码片段（简化版）：

```
1  c
2
3  运行
4
5
6
7

1  static int blocker_read(BlockDriverState *bs, int64_t offset,
2      uint8_t *buf, int size)
3  {
4      return -EIO; // 拦截读请求，返回I/O错误
5  }
6
7  static int blocker_write(BlockDriverState *bs, int64_t offset,
8      const uint8_t *buf, int size)
9  {
10     return -EIO; // 拦截写请求，返回I/O错误
11 }
12
13 static BlockDriver bdrv_blocker = {
14     .format_name = "blocker",
15     .bdrv_read = blocker_read,
```

```

16 .bdrv_write = blocker_write,
17 // 其他必要字段（如支持的协议、扇区大小等）
18 };
19
20 static void bdrv_blocker_init(void)
21 {
22     bdrv_register(&bdrv_blocker);
23 }
24

```

2. 触发 I/O 阻断（操作流程）

通过 QEMU Monitor（QMP/HMP 协议）动态激活阻断过滤器，覆盖 VM 的所有磁盘设备：

1. 获取 VM 的块设备列表：

通过 QMP 命令 `query-block` 获取 VM 当前挂载的所有磁盘设备（如 `virtio0`、`sata0` 等），确保无遗漏。

2. 为每个磁盘设备添加阻断过滤器：

使用 `blockdev-add` 命令，为每个磁盘设备添加 `blocker` 过滤器，示例 QMP 命令：

```

1 json
2
3
4
5
6
7
8
9
10
11
12 {
13     "execute": "blockdev-add",
14     "arguments": {
15         "node-name": "blocker-node",
16         "driver": "blocker",
17         "file": {
18             "node-name": "original-disk" // 原磁盘设备的node-name
19         }
20     }
21 }
22

```

执行后，所有针对该磁盘的 I/O 请求将被 `blocker` 拦截并返回 `-EIO`。

3. 阻断后终止 VM 的多级机制

I/O 被阻断后，VM 可能因磁盘错误进入异常状态，需通过三级机制确保终止：

1. 软关机（优先尝试）：

通过 QMP 命令 `system_powerdown` 发送 ACPI 关机信号，等待 VM 内部响应（如操作系统正常关机）。超时时间建议设为 30 秒（可根据 VM 配置调整）。

2. 强制关机（软关机失败时）：

若软关机超时，通过 QMP 命令 `destroy` 强制终止 VM。该命令直接触发 QEMU 内部的 VM 销毁流程，不受 I/O 状态影响（即使磁盘 I/O 阻塞，QEMU 仍会释放资源并退出）。

3. 进程级终止（最终保障）：

若 `destroy` 命令失败（极端情况，如 QEMU 进程僵死），通过操作系统信号终止进程：

- 。先发送 `SIGTERM`（优雅终止），等待 5 秒；
- 。若未退出，发送 `SIGKILL`（强制终止），确保进程被杀死。

三、关键保障机制

1. 全类型存储覆盖：

blocker 过滤器基于 QEMU 块设备抽象层实现，与底层存储类型（本地 / 网络、文件 / 块设备）无关，可拦截所有磁盘 I/O。

2. 避免进程僵死：

过滤器返回 **-EIO** 而非阻塞请求，确保 VM 进程处于可中断状态（R/S 状态），而非不可中断睡眠（D 状态），保证信号能正常传递。

3. 终止可靠性验证：

- 。对 **destroy** 命令的测试表明，即使块设备驱动异常，QEMU 仍会执行资源清理并退出；
- 。 **SIGKILL** 可直接终止任何用户态进程，是最终保障。

四、操作示例（简化流程）

1. 启动 VM 并记录其 PID（如 **12345**）和块设备 node-name（如 **virtio-disk0**）。

2. 通过 QMP 添加阻断过滤器：

```
1 bash
2
3
4
5
6

1 echo '{"execute": "blockdev-add", "arguments": {"node-name": "blocker0", "driver": "blocker", "file": {"node-name": "virtio-disk0"}}}' | socat - UNIX-CONNECT:/var/run/q
2
```

3. 尝试软关机：

```
1 bash
2
3
4
5
6

1 echo '{"execute": "system_powerdown"}' | socat - UNIX-CONNECT:/var/run/qemu/vm1.monitor
2
```

4. 30 秒后检查 VM 状态，若未关机，执行强制关机：

```
1 bash
2
3
4
5
6

1 echo '{"execute": "destroy"}' | socat - UNIX-CONNECT:/var/run/qemu/vm1.monitor
2
```

5. 若仍未终止，执行进程杀死：

```
1 bash
2
3
```

```
4
5
6
```

```
1 kill -15 12345; sleep 5; kill -9 12345
2
```

五、适用场景与局限性

- **适用场景**：需要紧急阻断 VM 磁盘操作（如恶意行为拦截、故障隔离）并强制终止的场景。
- **局限性**：
 - 需提前编译 `blocker` 过滤器到 QEMU（或作为模块加载）；
 - 对非磁盘 I/O（如网络、串口）无影响，若需全面阻断需额外处理。

通过上述方案，可实现对 QEMU/KVM 虚拟机所有磁盘 I/O 的可靠阻断，并确保 VM 能被强制终止，满足安全性与可控性要求。

方案六

在QEMU/KVM中阻止虚拟机I/O并确保成功关机或终止的方案

方案概述

本方案通过QEMU Monitor的 `drive_del` 命令阻止虚拟机所有磁盘I/O，结合virsh工具实现强制关机或进程终止，确保在I/O阻断后虚拟机能够可靠关闭。

详细步骤

1. 阻止虚拟机I/O

方法：使用QEMU Monitor的 `drive_del` 命令删除虚拟磁盘设备。

命令示例：

```
1 bash
```

```
virsh qemu-monitor-command <vm_name> --hmp "drive_del ide0-hd0"
```

- **说明**：
 - `<vm_name>`：虚拟机名称。
 - `ide0-hd0`：目标磁盘设备标识（根据实际配置调整，如 `virtio0`、`scsi0` 等）。
 - **效果**：立即移除磁盘设备，客户机所有I/O操作将触发错误，无法继续读写。

2. 触发虚拟机关机

正常关机（推荐）

前提：客户机需安装ACPI服务（如Linux需启用 `acpid`）。

命令：

```
1 bash
```

```
virsh shutdown <vm_name>
```

- 说明：发送ACPI关机信号，客户机操作系统配合完成关机流程。

强制终止进程（备用）

命令：

```
1 bash
```

```
virsh destroy <vm_name>
```

- 说明：直接终止QEMU进程，确保虚拟机立即停止，适用于正常关机失败场景。

3. 完整操作流程示例

```
1 bash
```

1. 阻止I/O

```
virsh qemu-monitor-command testvm --hmp "drive_del ide0-hd0"
```

2. 尝试正常关机

```
virsh shutdown testvm
```

3. 若未成功，强制终止

```
virsh destroy testvm
```

关键验证点

1. I/O阻断验证：

- 执行 `drive_del` 后，客户机日志应显示磁盘I/O错误（如 `Input/output error`）。
- 使用 `virsh dominfo <vm_name>` 检查状态，确认虚拟机处于 `shutdown` 或 `crashed` 状态。

2. 关机可靠性：

- 正常关机需确保客户机ACPI服务运行（如Linux执行 `systemctl start acpid`）。
- 强制终止通过 `virsh destroy` 直接中断进程，无需客户机配合。

注意事项

- 数据一致性：强制终止可能导致未提交数据丢失，建议提前暂停业务或使用快照。
- 设备标识：根据实际配置调整 `drive_del` 的目标设备（如 `virtio-blk0`、`scsi0`）。
- 客户机兼容性：Windows系统通常默认支持ACPI，部分Linux发行版需手动安装 `acpid`。

替代方案（可选）

- 使用cgroup冻结进程：

通过 `cgroup` 冻结虚拟机进程，暂停所有I/O：

```
1 bash
```

```
# 查找虚拟机进程PIDpgrep -f qemu-system-x86_64 # 冻结进程echo 1 > /sys/fs/cgroup/freezer/qemu/<vm_pid>/freezer.state
```

- 说明：需提前配置cgroup，适用于需要更细粒度控制的场景。

总结

本方案通过QEMU Monitor直接阻断I/O，结合virsh的关机命令，确保在阻断后虚拟机能够可靠关闭或终止。优先推荐正常关机流程，若失败则使用强制终止，兼顾安全性和效率。